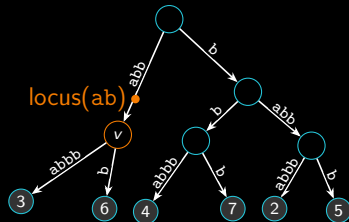


最小 suffixient 集合の準実時間構築

Dominik Köppl¹ Gregory Kucherov²

¹: 山梨大学 コンピュータ理工学

²: ギュスターヴ・エッフェル大学

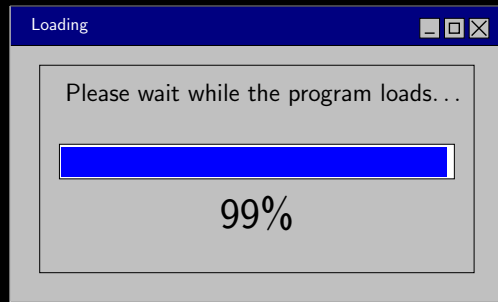


背景

- 古典的な問題：パターン照合，テキスト比較，データ圧縮など
- 大規模なテキスト集合に対して（例：DNA配列，リポジトリなど）

しかし：

- ほとんどのアルゴリズムは**オフライン**で動作：**いかなる**出力も生成される前に全体のテキストを必要とする
- 全入力処理が完了するまで待つ必要がある！
- 実世界の現象：バーが99%に達したときにアルゴリズムが本格的に動き始め，ユーザーは待たされる ...



オンラインアルゴリズム

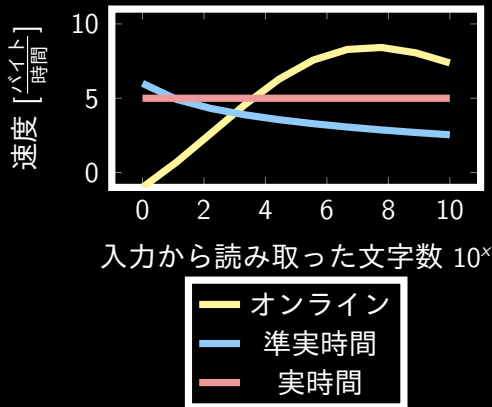
定義 (オンラインアルゴリズム)

新しい文字が到着するたびに，利用可能な解を漸進的に更新する

- ほとんどのオンラインアルゴリズムは平均化されている：1文字あたりの最悪時間は高くなる可能性がある！
- 実時間アルゴリズム：1文字あたりの最悪時間は定数

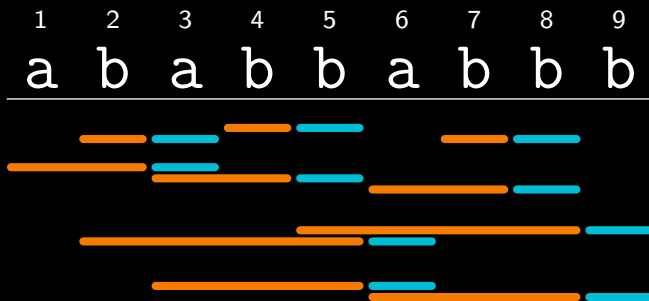
ここでは：

- 準実時間アルゴリズム：1文字あたりの最悪時間は入力サイズに対して対数的



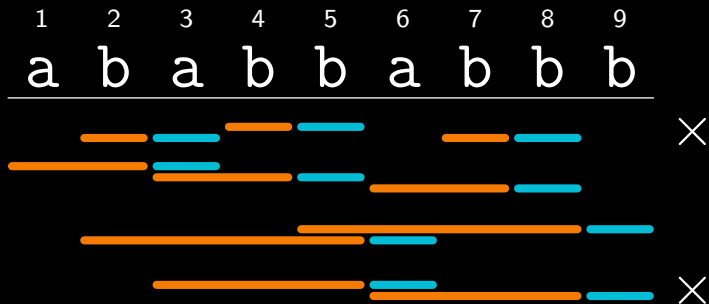
right extension

- ▮ $T[1..n]$: アルファベット Σ 上の文字列
- ▮ T の部分文字列 S と, $x, y \in \Sigma, x \neq y$ を考える
- ▮ $S \cdot x$ と $S \cdot y$ がともに T の部分文字列なら,
 $S \cdot x$ と $S \cdot y$ を right extension (RE) と呼ぶ



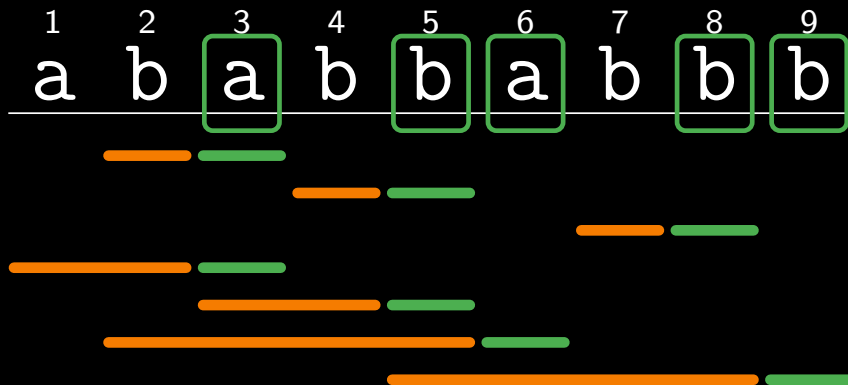
supermaximal right extension

- RE $S \cdot x$ を考える
 - $S \cdot x$ が他のどの RE の接尾辞でもないとき,
 $S \cdot x$ を **supermaximal right extension** (SRE) と呼ぶ
- ⇒ 任意の RE はある SRE の接尾辞である



suffixient set

- ▮ $P \subseteq [1..n]$ の位置集合 ($T[1..n]$ の長さは n)
- ▮ 各 RE の少なくとも1つの出現の終了位置を含むなら,
 P を suffixient set と呼ぶ



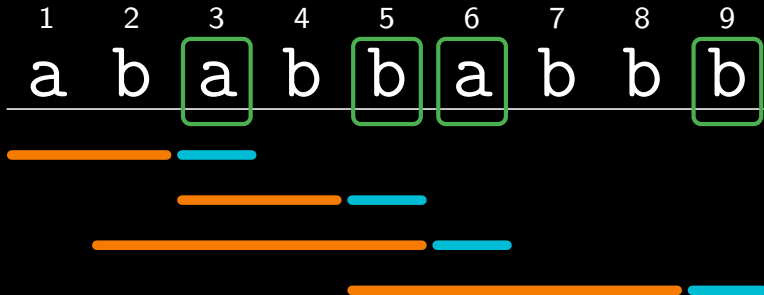
なぜ suffixient set が有用か

- suffixient set は圧縮索引の構成要素である Becker+'26
- 次の講演では, suffixient set をテキスト圧縮に利用する方法を示す
[柴田・坂内, LA 夏2026, 6号]

索引 + 圧縮

最小 suffixient set (SSS)

- $\chi(T) :=$ 最小 suffixient set (SSS) のサイズ
- $\chi(T) = |\text{SRE}(T)|$ (T に含まれる SRE の数)
- 構成 : 各 SRE について, その出現終了位置を1つ選ぶ



本研究の問題

入力

- アルファベット Σ 上の文字列 $T[1..n]$ をオンラインで読む
- 文字が右から左に1文字ずつ到着する： $T \leftarrow xT$.

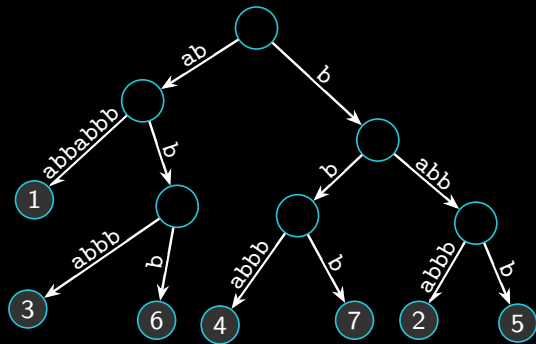
管理したいもの

- 接尾辞木 $ST(T)$
- SRE 集合 $SRE(T)$
- 右端出現位置からなる SSS

目標：各文字あたり最悪時間で保証する

接尾辞木

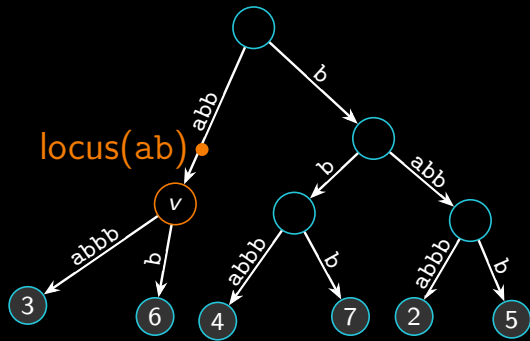
- 接尾辞木は, $T = \text{ababbabbb}$ のすべての接尾辞を表すコンパクトトライである



$T =$ 1 2 3 4 5 6 7 8 9
a b a b b a b b b

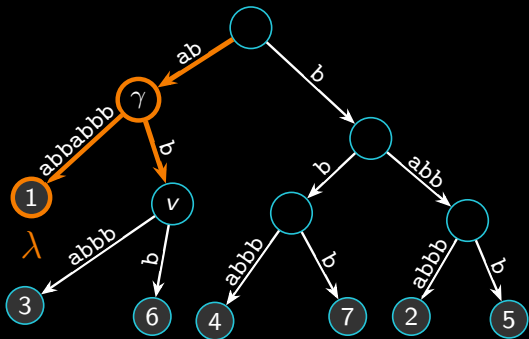
右から左入力と Weiner の接尾辞木

- まず $ST(T[2..])$ を持っているとする
- 新しい文字 $T[1]$ を左から追加して, $ST(T[1..])$ に更新する
- $\text{locus}(ab)$ の直下にある最上位のノード v へ向かう辺を分割する


$$T = \begin{matrix} & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ \begin{matrix} a & b & a & b & b & a & b & b & b \end{matrix} \end{matrix}$$

右から左入力と Weiner の接尾辞木

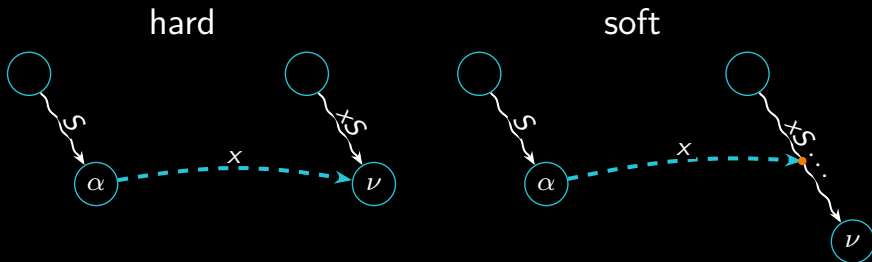
- まず $ST(T[2..])$ を持っているとする
- 新しい文字 $T[1]$ を左から追加して, $ST(T[1..])$ に更新する
- $\text{locus}(ab)$ の直下にある最上位のノード v へ向かう辺を分割する
- 新しい内部ノード $\gamma = \text{locus}(ab)$ を作り, 新しい葉 λ と v をその子として接続する



$T =$ 1 2 3 4 5 6 7 8 9
a b a b b a b b b

定義 : Weiner link

- $\text{locus}(S) = \alpha$ であるノード α を考える
- $\text{locus}(x \cdot S)$ が存在するなら, $W_x(\alpha)$ は $\text{locus}(x \cdot S)$ を指す.
- $W_x(\alpha)$ を α の x -link と呼ぶ
- $\text{locus}(x \cdot S)$ がノードなら, $W_x(\alpha)$ は **hard** である
- $\text{locus}(x \cdot S)$ が辺の途中にあるなら, $W_x(\alpha)$ は **soft** である



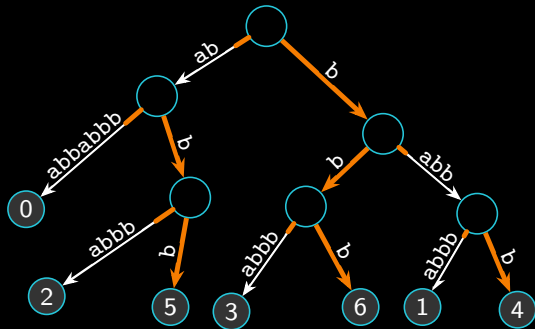
RE と接尾辞木の関係

- RE(T) は、接尾辞木の根から各辺の1文字目までのパス文字列の集合と一致する

補題

$S \cdot y$ を T の部分文字列とする。
 $S \cdot y$ が RE であることは、
次と同値である：

- locus(S) はノード α ,
- α は下降する y -edge を持つ



SRE の接尾辞木上の特徴付け

定理

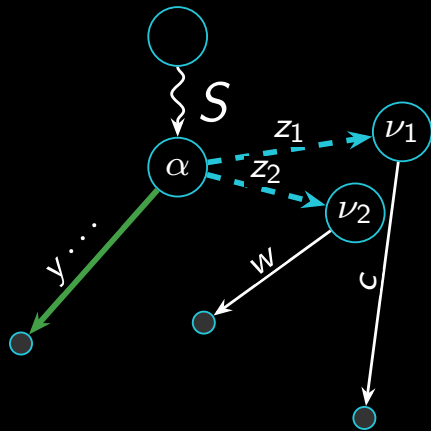
$S \cdot y$ を T の部分文字列とする.

$S \cdot y$ が SRE であることは,

次と同値である:

- $\text{locus}(S)$ はノード α ,
- α は下降する y -edge を持つ,
- 任意の hard link $W_z(\alpha) = \nu$ について, ノード ν は下降する y -edge を持たない

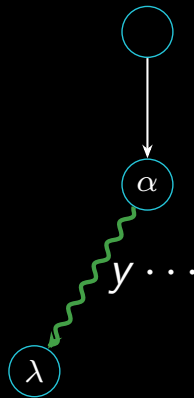
略記として, $S \cdot y$ を (α, y) と書く.



Weiner 更新1ラウンドで得られるノード

設定: T を $x \cdot T$ に更新する

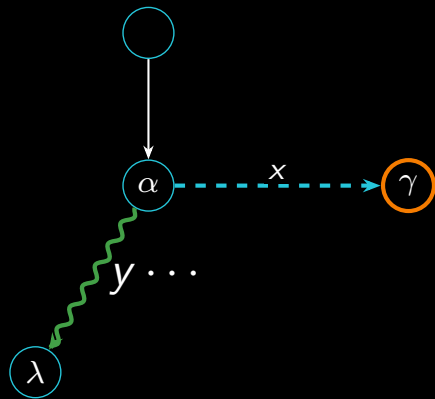
- λ : 更新前の T における最新の葉
- α : λ の祖先のうち hard x-link を持つ最下位のもの



Weiner 更新1ラウンドで得られるノード

設定: T を $x \cdot T$ に更新する

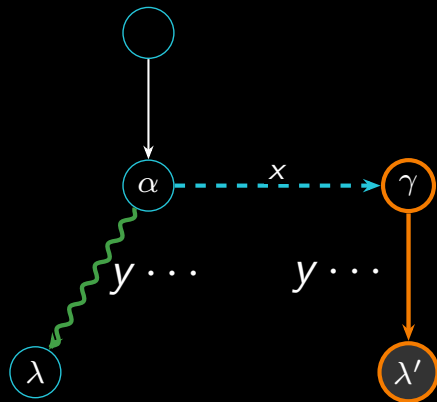
- λ : 更新前の T における最新の葉
- α : λ の祖先のうち hard x -link を持つ最下位のもの
- Weiner 探索は α と $\gamma = W_x(\alpha)$ を見つける



Weiner 更新1ラウンドで得られるノード

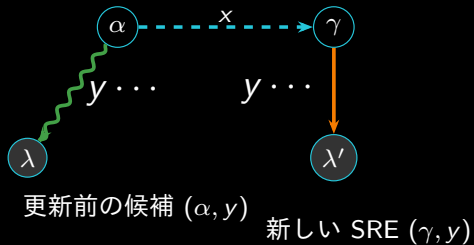
設定: T を $x \cdot T$ に更新する

- λ : 更新前の T における最新の葉
 - α : λ の祖先のうち hard x-link を持つ最下位のもの
 - Weiner 探索は α と $\gamma = W_x(\alpha)$ を見つける
 - 新しい葉 λ' が追加される.
- ⇒ SRE 更新に必要な主なノードはここで「ただで」得られる



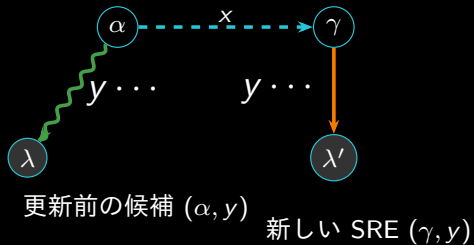
hard link の場合

■ $W_x(\alpha) = \gamma$ が既存ノード



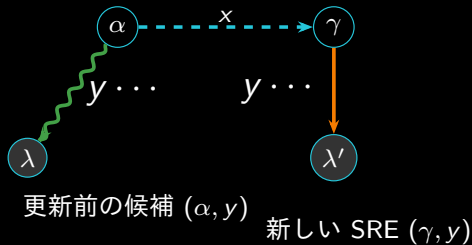
hard link の場合

- $W_x(\alpha) = \gamma$ が既存ノード
- γ に新しい y -edge が生まれる



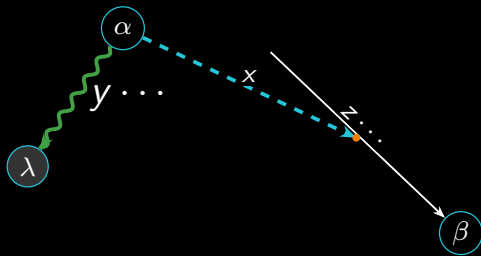
hard link の場合

- $W_x(\alpha) = \gamma$ が既存ノード
- γ に新しい y -edge が生まれる
- 新しい SRE (γ, y) を追加する
- (α, y) が SRE なら削除する



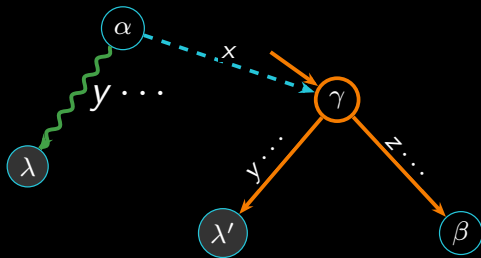
soft link の場合

- ▮ $W_x(\alpha)$ が辺の途中を指す.



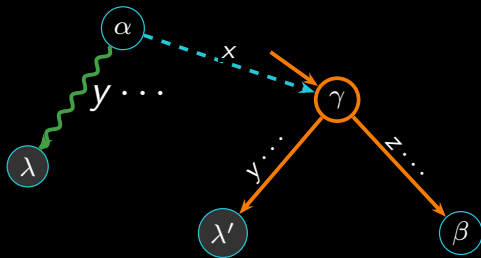
soft link の場合

- $W_x(\alpha)$ が辺の途中を指す.
- Weiner 更新で辺を分割し, 新しい内部ノード γ を作る



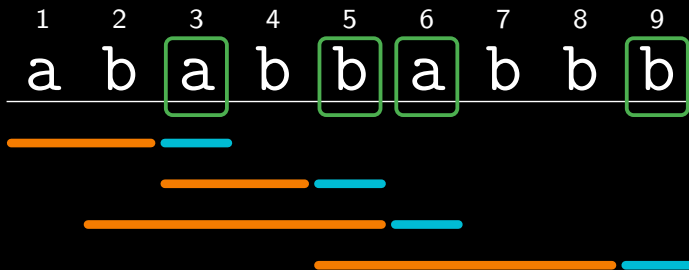
soft link の場合

- $W_x(\alpha)$ が辺の途中を指す.
- Weiner 更新で辺を分割し, 新しい内部ノード γ を作る
- γ の 2 本の下降辺から, (γ, y) と (γ, z) が新しい SRE になる
- (α, y) または (α, z) が SRE なら削除する



右端位置 SSS の管理

- 各 SRE は接尾辞木の一意な辺に対応する
- その SRE の右端出現位置を1つ、例えば右端のものとして選ぶ
- SRE の追加・削除は、対応するビットを反転するだけでよい



アルゴリズムの要約

1. Weiner の準実時間接尾辞木更新を実行する
2. その更新から α, γ, λ' と辺ラベルの先頭文字 y, z を読む
3. 追加・削除すべき $O(1)$ の SRE だけを更新する
4. SRE 変更と同時に右端位置のビットベクトルを更新する

まとめ

定理

右から左入力において、SRE 集合と、SRE の右端出現位置からなる SSS は、各文字あたり最悪 $O(t_{ST})$ 時間で管理できる

- 多項式サイズ整数アルファベットでは $t_{ST} = O(\log^2 \log n)$ であり、
- $\sigma = O(\log^{1/4} n)$ なら $O(\log \log n)$ である

- SSS は文字列の反復性を測る位置集合であり、SRE と同じ数だけで十分
- 右から左では Weiner の接尾辞木更新と相性がよい
- 1ラウンドで変化し得る SRE は定数個だけである
- したがって、接尾辞木の準実時間更新から SRE/SSS の準実時間管理が得られる

ご清聴ありがとうございました

SSS 計算の既存研究

研究	結果
Depuydt+23	構築時間は未解決
Cenzato+24	線形時間
Bonizzoni+26	線形時間
Olivares-Navarro'26	線形時間
Fujimaru+'26	1文字あたり償却 $O(\lg n)$ 時間
本講演	1文字あたり最悪 $O(\lg^2 \lg n)$ 時間

⇒ 既存研究は主に静的構築であり，本講演はオンライン管理へ進む。